

## VERIFICA PRATICA — RECUPERO

Web Service REST + Client | PHP OOP + MySQL

Candidato/a: \_\_\_\_\_ Data: \_\_\_\_\_

### Contesto e obiettivo

Lo studente deve realizzare un'applicazione web per la gestione di un **catalogo di videogiochi**. Il catalogo permette di consultare i titoli disponibili, organizzati per sviluppatore e genere, e raggruppati in **bundle promozionali**. Il progetto è diviso in un **Web Service REST in PHP OOP** e un **client HTML/CSS/JavaScript**. Il database da utilizzare è **videogiochi** (file SQL allegato).

### Schema del Database

Database: videogiochi

| Tabella      | Chiave Primaria | Campi principali                                      |
|--------------|-----------------|-------------------------------------------------------|
| sviluppatore | idSviluppatore  | nomeSviluppatore, nazione                             |
| genere       | idGenere        | nomeGenere, descrizione                               |
| videogioco   | idVideogioco    | titoloGioco, anno, idSviluppatore (FK), idGenere (FK) |
| bundle       | idBundle        | nomeBundle, tema, prezzo                              |
| composizione | nroProgressivo  | idBundle (FK), idVideogioco (FK)                      |

Un videogioco appartiene a uno sviluppatore e a un genere; un bundle promozionale raggruppa più videogiochi attorno a un tema comune tramite la tabella composizione.

### Punto 1 — Web Service REST (PHP OOP)

#### 1.1 Endpoint richiesti

| Metodo | URL                | Descrizione                                                                          |
|--------|--------------------|--------------------------------------------------------------------------------------|
| GET    | /videogiochi       | Restituisce tutti i videogiochi con sviluppatore e genere (JOIN)                     |
| GET    | /videogiochi/{id}  | Restituisce il dettaglio di un singolo videogioco                                    |
| POST   | /videogiochi       | Inserisce un nuovo videogioco                                                        |
| DELETE | /videogiochi/{id}  | Elimina un videogioco                                                                |
| GET    | /bundle?gioco={id} | Restituisce i bundle che contengono il videogioco indicato (filtro per query string) |

Tutte le risposte devono essere in formato JSON con header Content-Type: application/json. La modifica (PUT) non è richiesta in questa prova.

## 1.2 Struttura OOP minima

Il Web Service deve essere strutturato con almeno le seguenti classi:

- Database.php — gestione connessione
- Videogioco.php — operazioni CRUD sulla tabella videogioco
- Bundle.php — operazioni di lettura sulla tabella bundle

## Punto 2 — Client Web (HTML + JS)

### 2.1 Home Page — Catalogo dei videogiochi

La pagina principale deve:

- Mostrare una tabella con tutti i videogiochi presenti nel DB, comprensivi di nome sviluppatore e genere
- Per ogni riga esporre due link/pulsanti: Elimina videogioco e Vedi bundle
- Includere un form o un link per l'inserimento di un nuovo videogioco
- Tutte le operazioni devono avvenire tramite chiamate AJAX al Web Service (fetch API o XMLHttpRequest)

### 2.2 Pagina Bundle di un Videogioco

Raggiungibile dal link «Vedi bundle» della Home Page. La pagina deve:

- Richiamare l'endpoint GET /bundle?gioco={id} e visualizzare i bundle che contengono il videogioco selezionato
- Per ogni bundle mostrare: nome, tema, prezzo
- Includere un link o pulsante per tornare alla Home Page

## Criteri di Valutazione

| Criterio                                                                 | Punti max | Punti assegnati |
|--------------------------------------------------------------------------|-----------|-----------------|
| Correttezza e completezza degli endpoint REST                            | 4         |                 |
| Struttura OOP e qualità del codice PHP                                   | 3         |                 |
| Home page: tabella videogiochi con sviluppatore/genere e operazioni AJAX | 5         |                 |
| Pagina bundle di un videogioco                                           | 4         |                 |
| Gestione errori HTTP e risposte JSON corrette                            | 2         |                 |
| Cura dell'interfaccia e usabilità                                        | 2         |                 |
| <b>TOTALE</b>                                                            | <b>20</b> |                 |

## Note e avvertenze

- Il codice deve essere consegnato tramite la cartella personale sul server di laboratorio.
- Non è consentito l'utilizzo di framework PHP. È ammesso Bootstrap o CSS personalizzato lato client.
- Web Service e client devono risiedere in cartelle separate (es. api/ e client/).
- La funzionalità PUT (modifica videogioco) non è richiesta in questa prova.
- È vietata la comunicazione con altri studenti durante la prova.